# Klassifikation anhand von neuronalen, erlernten Konzepten

Ilona Eisenbraun
s0561762
10. Oktober 2022

Hochschule für Technik und Wirtschaft Berlin
Fachbereich 4: Informatik Kommunikation und Wirtschaft
Studiengang: Angewandte Informatik Master
Kurs: Independent Coursework 2
Prüfer: Prof. Dr. rer. nat. Peter Hufnagl

# Inhaltsverzeichnis

1 Einleitung 4

2 Implementierung 4
2.1 Überblick . . . . . 5
2.2 Konzept-Attributierung . . . . . 5
2.2.1 Layerübergreifende Attributionen . . . . . 7
2.2.2 Bildregionenspezifische Attributionen . . . . . 8
2.2.3 Fragmentierung der Attributionen . . . . . 9
2.3 Konzept-Visualisierung . . . . . 9

3 Ergebnisse 11
3.1 Layerübergreifende Attributionen . . . . . 11
3.2 Fragmentierung der Attributionen . . . . . 13
3.3 Bildregionenspezifische Attributionen . . . . . 14

4 Fazit und Ausblick 14

# Tabellenverzeichnis

1 Übersicht der enstandenen Module, ihrer Methoden und die entsprechenden Funktionalitäten . . . . . 5

# Abbildungsverzeichnis

1 Dictionary-Ausgabe der layerübergreifenden wichtigsten Konzepte, gemittelt über Votings aller Samples . . . . . 12
2 Dictionary-Ausgabe der Relevanzen für die wichtigsten Konzepte, gemittelt über alle Samples . . . . . 12
3 Konzept-Attributierungen der Layer features .8, features .18, features .22, classifier .3, classifier .6 für die Datei "Mitosis_152_-{X=437293,Y=166519}.jpg" . . . . . 13
4 Dictionary-Ausgabe der für die Konzepte wichtigsten Unterkonzepte, gemittelt über alle Samples . . . . . 14
5 Dictionary-Ausgabe der lokalen wichtigsten Konzepte, gemittelt über alle Samples . . . . . 14

# 1 Einleitung

Die nachfolgende Arbeit beschäftigt sich mit der neuronalen Klassifikation dank gelernter Konzepte, die sich in Netzwerken wiederfinden. Hierzu wurde im "Independent Coursework I" zuvor ein "Pruning eines neuronalen Netzwerks anhand von KI-Erklärbarkeit" ausgearbeitet. Anhand der über die Layerwise Relevance Propagation (LRP) [1] ermittelten Relevanzen, die die Erklärbarkeit von Netzwerkentscheidungen transparenter machen, konnte das Netzwerk um Einiges verkleinert werden. Dagegen die durch das ProtoPNet [2] erreichte Erklärbarkeit sorgte für das Offenlegen von Prototypen, die aussagekräftig zur Klassifizierung beitragen. Diesbezüglich konnte das Pruning dieser für eine genauere Distinktion zwischen den Klassen hinreichend sein.

Auf diesem Hintergrund und in Anlehnung an das "Forschungsprojekt B" werden in dieser Arbeit dem Forschungsprojekt ähnliche Rahmenbedingungen geschaffen: Anhand des am CBMI [3] enstandenen Mitosen-Klassifikators[1] besteht der Fokus auf der Untersuchung der Klassifikations-Konzepten, die näher inspiziert werden können.

In Anbetracht dessen wird auf dem in dem "Forschungsprojekt B" vorgestellten Ansatz zum Auffinden genutzt: Die bereits vorgestellte Concept Relevance Propagation (CRP) [4] eignet sich, um relevante und irrelevante Konzepte hinsichtlich einer Klassifikation zu identifizieren. Weiter kann mittels dem Python-Package Zennit-CRP [5] dieses Vorhaben praktisch umgesetzt werden.

Im Folgenden wird daher untersucht, wie diese erlernten Konzepte selektiv zu einer Mitosen-Klassifikation beitragen - und wie sich diese nach vorherigem, dem Forschungsprojekt entstandenem Pruning verändert.

# 2 Implementierung

Um das Vorhaben umsetzen zu können, sind einige Python-Bibliotheken notwendig. Diese sind im Folgenden mitsamt der Versionsnummern aufgeführt:

- Zennit 0.4.7 [6]
- Zennit-CRP 0.4.2 [5]
- Pytorch 1.12.0 [7]
- NumPy 1.22.4 [8]
- Pillow 9.2.0 [9]
- OpenCV 4.6.0 [10]

Zennit und Zennit-CRP sind hier die für die Attribution der Konzepte innerhalb der Layer entscheidend. Dagegen Pytorch und NumPy werden für die Erstellung, Berechnung

[1] Hierzu werden ebenso das zugehörige `state_dict_path`, der Datensatz `examples_ds_from_MP.train.HTW.train` und der zur Verfügung gestellte Modell-Quellcode genutzt

und Konvertieren von Tensoren genutzt. Pillow und OpenCV schließlich helfen, grafische Elemente zu lesen und zu verarbeiten.

## 2.1 Überblick

Im Rahmen dieses "Independent Coursework II" als auch dem "Forschungsprojekt B" enstand ein Repository, das auf Github verfügbar ist [11]. Die daraus entstandene Pipeline basiert auf mehreren Modulen. Die Tabelle 1 veranschaulicht diese näher:

| Modul | Methode | Rolle |
|---|---|---|
| `__main__.py` | • `init_datasets`<br>• `dataset_split`<br>• `finetune_model`<br>• `evaluate_model` | • Erstellung eines Trainings- und Validierungs-Dataloaders<br>• Aufteilung des Datensatzes in Subsets<br>• Adjustierung des Modells nach vorherigem Konzept-Pruning<br>• Auswertung der Modell-Performance auf Beispieldatensatz |
| `concept_attribution.py` | • `local_attribute_imgs`<br>• `latent_attribute_imgs`<br>• `retrieve_prunable_true_concepts`<br>• `retrieve_prunable_concept_diff` | • An der Eingabe verortete Ermittlung der relevantesten Konzepte<br>• Layerübergreifende Ermittlung der relevantesten Konzepte<br>• Auszug der unwichtigen "True"-Konzepte als Dictionary<br>• Auszug der Konzept-Differenz der Klassen als Dictionary |
| `concept_pruning.py` | • `prune_selected_concepts` | • Entfernen der Netzwerk-Filter anhand des Pruning-Dictionaries |
| `concept_visualization.py` | • `gen_heatmaps`<br>• `gen_broadcast_heatmaps`<br>• `gen_heatmaps_starting_layer`<br>• `plot_imgs` | • Erstellung Heatmap-Attributionen der Konzepte<br>• Erstellung Heatmap-Attributionen für mehrere Klassen<br>• Erstellung Heatmap-Attributionen ab einem Start-Layer<br>• Grafische Darstellung der Heatmaps mitsamt des Originals |
| `utils.py` | • `init_attribution_vars`<br>• `measure_module_sparsity`<br>• `measure_global_sparsity`<br>• `take`<br>• `get_max_ids`<br>• `nested_dict_iter` | • Initialisierung von Variablen für die Konzept-Attribution<br>• Bemessung der layerspezifischen Parameter-Spärlichkeit<br>• Bemessung der globalen Parameter-Spärlichkeit<br>• Extraktion der ersten $n$ Elemente einer Liste<br>• Extraktion der Layer und Konzepte mit höchsten Votes<br>• Extraktion von Schlüsseln und Werten eines geschachtelten Dictionaries |

Tabelle 1: Übersicht der enstandenen Module, ihrer Methoden und die entsprechenden Funktionalitäten

Die Module sind hier hinsichtlich ihrer Aufgabenbereiche gegliedert. Obwohl einige Methoden einige Importe aus anderen Modulen nutzen, sind die Abhängigkeiten weitestgehend reduziert. Im Folgenden werden die einzelnen Module und die Funktionalitäten näher beschrieben.

## 2.2 Konzept-Attributierung

Um Konzepte als auch Subkonzepte verstehen zu können, kann hier mittels Zennit-CRP [5] die relevanzbasierte Attribution bestimmt werden. In diesem Zusammenhang

ist Zennit [6] für die einhergehende, auf die Layerwise Relevance Propagation (LRP) basierende Propagierung wichtig. Zennit-CRP baut hierbei stark auf Zennit und die mitgelieferten Module auf. Es werden entsprechend zuerst Variablen gesetzt, wie im Algorithmus 1 zu sehen ist.

```
cc = ChannelConcept()
composite = EpsilonPlusFlat([SequentialMergeBatchNorm()])
attribution = CondAttribution(model)
layer_names = get_layer_names(model, [torch.nn.Conv2d, torch.nn.Linear])
```

Algorithmus 1: Initialisierung der Variablen für die Berechnung der Konzept-Relevanzen (Auszug aus der Methode `init_attribution_vars()`)

Um nun einhergehend Attributionen von Konzepten innerhalb des Netzwerks auszumachen, wird die Menge an Samples eines Datensatzes jeweils nach den Relevanz-Attributionen untersucht (Alg. 2).

```
attr = attribution(sample, condition, composite, record_layer=layer_names)
for model_layer in layer_names:
    name = "model_ft"
    single_layer = model_layer[model_layer.index('.') + 1 : ]
    for module_name, _ in model._modules[name].named_modules():
        if single_layer == module_name:
            rel_c = cc.attribute(attr.relevances[model_layer], abs_norm=True)
            concept_ids = torch.argsort(rel_c[0], descending=descending)[:10]
```

Algorithmus 2: Ermittlung der Relevanzen an jedem einzelnen Sample (Auszug aus der Methode `latent_attribute_imgs()`)

Über den Datensatz iterierend wird für alle Layer die Attribution `attr` bestimmt, um diese dann anhand ihrer `relevances` in den einzelnen, im Netzwerk vorhandenen Layer (`model_layer`) zu verorten. Davon ausgehend werden die Attributionen ihrer Wichtigkeit nach sortiert, und die zehn relevantesten Konzepte abgespalten.

Nachfolgend werden die zehn relevantesten Konzepte über alle Samples im Datensatz gesammelt - und anhand eines Zählers jeweils die Einträge für das Layer und die Konzept-ID inkrementiert (siehe Alg 3).

```
concept_atlas={}
for concept in concept_ids:
    concept = concept.cpu().item()
    if ( (single_layer not in concept_atlas)
       or
       (concept not in concept_atlas[single_layer])) :
        concept_atlas.setdefault(single_layer, dict())[concept] =1
    else:
        concept_atlas[single_layer][concept]+=1
```

Algorithmus 3: Erstellung eines `concept_atlas` nach einem Voting über die Konzepte über alle Samples (Auszug aus der Methode `latent_attribute_imgs()`)

### 2.2.1 Layerübergreifende Attributionen

Um Konzepte, die für die "True"-Klasse entscheidend global umfassend zu ermitteln, wird hier zuerst die Kondition gesetzt ($y = 1$). Über alle Layer iterierend - und ebenso auch über die "True"-Samples - lassen sich die Attributionen mit `attribution(sample, conditions, composite, record_layer=layer_names)` bestimmen. Der Algorithmus 4 zeigt hier auf, wie die fünf wichtigsten Konzepte für jedes Layer und für jedes Sample extrahiert werden.

```
conditions = [{'y': [1]}]
concept_atlas={}
for single_layer in layer_names:
    layer_concept_dict={}
    for file in os.listdir(path):
        path_to_file = os.path.join(path,file)
        image = Image.open(path_to_file).convert('RGB')
        sample = transform(image).unsqueeze(0)
        sample.requires_grad = True
        sample = sample.to(device)
        attr = attribution(sample, conditions, composite, record_layer=layer_names)
        rel_c = cc.attribute(attr.relevances[single_layer], abs_norm=True)
        indices = torch.argsort(rel_c[0], descending=True)[:5]
        for concept in indices:
            concept = concept.cpu().item()
            if ( (concept not in layer_concept_dict)) :
                layer_concept_dict[concept] =1
            else:
                layer_concept_dict[concept]+=1
```

```
    layer_concept_dict= dict( sorted(layer_concept_dict.items(),
                                key=operator.itemgetter(1),reverse=True))
    layer_concept_dict  = {k: layer_concept_dict[k] for k
                                in list(layer_concept_dict)[:5]}
    concept_atlas[single_layer] = layer_concept_dict
```

Algorithmus 4: Bestimmung der wichtigsten Konzepte zur Mitosen-Klassifikation im latenten Raum über alle Samples (Auszug aus der Methode `latent_attribute_imgs()`)

Über die Menge an untersuchten mitotischen Beispielen der die "True"-Klasse wird so über einen Zähler ein Voting über die wichtigsten Konzepte erstellt. Davon ausgehend werden für jedes Layer die fünf bedeutendsten Konzepte herausgefiltert, um diese als `concept_atlas` weiterverarbeiten zu können.

### 2.2.2 Bildregionenspezifische Attributionen

Ähnlich der globalen Ermittlung von Attributionen lassen sich spezifisch ausgewählte Bildregionen bezüglich ihrer Bedeutung für die Klassifikation analysieren. Auch hier werden Konditionen festgelegt (Alg.5):

```
conditions = [{single_layer: [id], 'y': [1]} for id in np.arange(0, features[ind])]
for file in os.listdir(path):
    imgs={}
    mask = torch.zeros(512, 512)
    mask[200:300, 200:300] = 1
    path_to_file = os.path.join(path,file)
    image = Image.open(path_to_file).convert('RGB')
    sample = transform(image).unsqueeze(0)
    sample.requires_grad = True
    sample = sample.to(device)
    rel_c = []
    for attr in attribution.generate(sample, conditions, composite,
                                record_layer=layer_names, batch_size=2):
        masked = attr.heatmap * mask[None, :, :]
        rel_c.append(torch.sum(masked, dim=(1, 2)))
    rel_c = torch.cat(rel_c)
    indices = torch.argsort(rel_c, descending=True)[:5]
```

Algorithmus 5: Ermittlung für Regionen im Eingabebereich zur Klassifikation beitragenden Konzepte (Auszug aus der Methode `local_attribute_imgs()`)

Hinsichtlich der möglichen Kandidaten, die als Konzept-Encoder in Frage kommen, wird hier eine Liste mit Indizes aller, je Layer vorhandenen Filter erstellt. Weitergehend wird eine Maske erstellt, deren mittiges Kernstück markiert ist - dies liegt der Tatsache zugrunde, dass bei den vorliegenden Bildern die Mitosen vor Allem im Zentrum liegen.

Entsprechend werden über einen Generator heatmaps mit `attr.heatmap` erzeugt, und mit der Maske multipliziert. Die ermittelten Relevanzen und Konzepte werden anschließend wieder auf die fünf wichtigsten reduziert.

### 2.2.3 Fragmentierung der Attributionen

Auf ähnliche Weise lassen sich die Konzepte in ihre Unterkonzepte zerlegen. Mit der gesetzten Zielvariable wird, über alle Layer iterierend, jeweils der Vorgänger ermittelt. Die Konzepte, die in einem Dictionary für das Vorgänger-Layer festgehalten worden sind, bilden die Konditionen: Der Algorithmus 6 veranschaulicht dies.

```
if(single_layer != 'model_ft.features.0'):
    prev_layer = key_layer_list[ind-1]
    keys_list = [key for key in concept_atlas[single_layer]]
    conditions = [{"y": [1], single_layer: [keys_list[0]]}]
```

Algorithmus 6: Generierung von Konditionen für Konzepte von Vorgängerlayern (Auszug aus der Methode `decompose_attribute_imgs()`)

Um nun über eine Rückpropagierung Bestandteile des Konzepts zu finden, wird über das `record_layer` der Ausgangspunkt, beziehungsweise das vorherige Layer respektive der Konditionen angegeben (Alg. 7).

```
attr = attribution(sample, conditions, composite, record_layer=[prev_layer])
rel_c = cc.attribute(attr.relevances[prev_layer], abs_norm=True)
```

Algorithmus 7: Rückpropagierung von Relevanzen ausgehend vom aktuellen Layer (Auszug aus der Methode `decompose_attribute_imgs()`)

## 2.3 Konzept-Visualisierung

Nach Extraktion der gewünschten Konzepte lassen sich diese weitergehend visualisieren. Hierfür wird in der Methode `gen_heatmaps` über ein übergebenes Dictionary iteriert, dessen Schlüssel die Netzwerklayer sind. Für jeden Schlüssel werden die Werte mit den Konzept-Indices für die Konditionen `conditions` befüllt. Diese werden genutzt, um über die `attribution` eine Reihe an Heatmaps zu dem Sample zu generieren. Diese werden mittels `zimage` zu einer Reihe an `PIL.Images` umgeformt - und dann in einem Dictionary abgelegt (Algorithmus 8).

```
conditions = [{'model_ft.'+layer: [torch.tensor(id).to(device)], 'y': [1]}
                                    for id in prunable_channels[layer]]
if len(conditions):
    heatmap, _, _, _ = attribution(sample, conditions, composite)
    img=(zimage.imgify(heatmap, symmetric=True,
                grid=(1, len(prunable_channels[layer]))))
    heatmaps[layer]=img
```

Algorithmus 8: Generierung von Heatmaps in Abhängigkeit vom Layer, dem Sample und den Konzepten (Auszug aus der Methode `gen_heatmaps()`)

Dem zugrundeliegend können die Heatmaps durch die Methode `plot_imgs` grafisch dargestellt werden. Abhängig von dem der Methode übergebenen Boolean `pruned_bool` ändert sich die `flag`, und es werden Initalwerte gesetzt. Über die Heatmaps des Dictionaries iterierend werden diese konvertiert und mit einem Rand befüllt.

Graduell wird die Schriftgröße an die anteilige, zwanzigprozentige Weite des Bildes angepasst, wie im Algorithmus 9 zu sehen ist. Über `PIL.ImageDraw.Draw` wird die Heatmap(-reihe) dann gerendert. Entsprechend der Estimation zur Größe der Schriftart wird der Text zur oberen Mitte des `Images` verschoben. Um die Seitenverhältnisse aller Heatmaps zu wahren, wird die Heatmap an die geringste Breite `min_img_width` aller Heatmaps angepasst. Schlussendlich wird die Gesamthöhe um die neuen Heatmaps erweitert.

```
heatmap = heatmaps[key]
heatmap.convert('RGB')
heatmap = ImageOps.expand(heatmap, border=int(0.2*heatmap.size[1]),
                                            fill=(255,255,255))
while font.getsize(key)[0] < img_fraction*heatmap.size[0]:
    fontsize += 1
    font = ImageFont.truetype(fontfile, fontsize)
font = ImageFont.truetype(fontfile, fontsize)

d = ImageDraw.Draw(heatmap)
w,h = font.getsize(key)
d.text((int((heatmap.width-w)/2),0), key, fill=(0,0,0),font=font)
if heatmap.width > min_img_width:
    heatmaps[key] = heatmap.resize((min_img_width,
                            int(heatmap.height / heatmap.width * min_img_width)),
                                Image.ANTIALIAS)
total_height += heatmaps[key].height
```

Algorithmus 9: Generierung von Heatmaps in Abhängigkeit vom Layer, dem Sample und den Konzepten (Auszug aus der Methode `plot_imgs()`)

Nun wird auch das Original zum Plot hinzugefügt. Diese Prozedur verläuft ähnlich; Hier wird jedoch ein neues `Image` erzeugt, und mittels `paste` zuerst weiß gefüllt. Anschließend werden die Heatmaps nach und nach in jede Reihe gefügt, um abschließend das Sample, welches auf die Größe der anderen Bilder skaliert wurde zu transferieren (Alg. 10).

```
wpercent = (min_img_width/float(sample.size[0]))
hsize = int((float(sample.size[1])*float(wpercent)))

img_merge = Image.new(heatmap.mode, (min_img_width+sample.width+
                                     (2*int(0.1*sample.size[1])),
                                     total_height)).convert('RGB')
img_merge.paste((255,255,255), [0,0,min_img_width+sample.width+
                                2*int(0.1*sample.size[1]),total_height])
y = 0
for image in heatmaps.values():
    img_merge.paste(image, (0, y))
    y += image.height
img_merge.paste(sample, (sample.width,
                         int(sample.height/2)-2*int(0.1*sample.size[1])))
```

Algorithmus 10: Befüllen eines `Pil.Image` anhand der Heatmaps und des Originals (Auszug aus der Methode `plot_imgs()`)

# 3 Ergebnisse

Der Implementierung folgend konnten einige Resultate hinsichtlich der Extraktion von für die Klassifikation wichtigen Konzepten erzielt werden. Im Folgenden werden hier die unterschiedlichen Methodiken genauer betrachtet.

## 3.1 Layerübergreifende Attributionen

Für das Auffinden von Konzepten im latenten Raum des Netzwerks wurde ein Dictionary extrahiert. Dieses hält die Votings der wichtigsten Konzepte in jedem Layer über alle Beispielbilder fest. Die Abbildung 1 veranschaulicht, wie sich dieses zusammensetzt; für die jeweils fünf höchsten Zählstände werden die Konzepte als `key`—`value`-Paar herausgeschnitten.

```
'model_ft.features.0': {30: 40, 29: 40, 24: 40, 26: 40, 19: 40},
'model_ft.features.4': {126: 40, 121: 40, 112: 40, 106: 40, 101: 40},
'model_ft.features.8': {91: 30, 63: 27, 88: 23, 85: 20, 87: 14},
'model_ft.features.11': {240: 13, 204: 13, 63: 12, 233: 10, 93: 10},
'model_ft.features.15': {394: 8, 267: 8, 376: 8, 254: 8, 243: 7},
'model_ft.features.18': {269: 10, 254: 9, 437: 9, 225: 8, 248: 8},
'model_ft.features.22': {94: 25, 306: 24, 358: 21, 344: 12, 156: 8},
'model_ft.features.25':  {370: 40, 193: 40, 17: 38, 133: 34, 317: 22},
'model_ft.classifier.0': {3257: 40, 949: 40, 2182: 40, 1421: 39, 1191: 29},
'model_ft.classifier.3': {3641: 40, 1091: 40, 863: 40, 3137: 40, 1464: 40}
```

Abbildung 1: Dictionary-Ausgabe der layerübergreifenden wichtigsten Konzepte, gemittelt über Votings aller Samples

Entsprechend können die prozentualen Relevanzen der einzelnen Konzepte in den Layern ermittelt werden; zuerst einzelne Kontributionen zur Klassifikation werden über alle Bilder gemittelt (siehe Abb. 2).

```
'model_ft.features.0': {30: 0.0, 29: 0.0, 24: 0.0, 26: 0.0, 19: 0.0},
'model_ft.features.4': {126: 0.0, 121: 0.0, 112: 0.0, 106: 0.0, 101: 0.0},
'model_ft.features.8': {91: 0.0, 63: 0.0, 88: 0.0, 85: 0.0, 87: 0.0},
'model_ft.features.11': {240: 0.0, 204: 0.0, 63: 0.0, 233: 0.0, 93: 0.0},
'model_ft.features.15': {394: 0.0001, 267: 0.0001, 376: 0.0002, 254: 0.0, 243: 0.0},
'model_ft.features.18': {269: 0.0003, 254: 0.0, 437: 0.0004, 225: 0.0001, 248: 0.0},
'model_ft.features.22': {94: 0.0158, 306: 0.0109, 358: 0.0021, 344: 0.0143, 156: 0.0005},
'model_ft.features.25': {370: 1.0575, 193: 0.8685, 17: 0.7052, 133: 0.4667, 317: 0.2513},
'model_ft.classifier.0': {3257: 0.3553, 949: 0.3275, 2182: 0.3204, 1421: 0.3088, 1191: 0.2223},
'model_ft.classifier.3': {3641: 0.3184, 1091: 0.2646, 863: 0.2565, 3137: 0.2487, 1464: 0.2396}
```

Abbildung 2: Dictionary-Ausgabe der Relevanzen für die wichtigsten Konzepte, gemittelt über alle Samples

Zur Visualisierung der für die Klassifikation relevanten Konzepte lassen diese sich innerhalb ihrer Layer respektive der Zielvariable $y = 1$ als Heatmap darstellen. Exemplarisch zeigt die Abbildung 3, wie hier für die Bilddatei "Mitosis_152_-{X=437293,Y=166519}.jpg" für ein unteres, ein mittleres und ein höheres Layer des Feature-Extraktors und zwei lineare Layer des Klassifikators die Konzepte Attributionen im latenten Raum ausmachen.

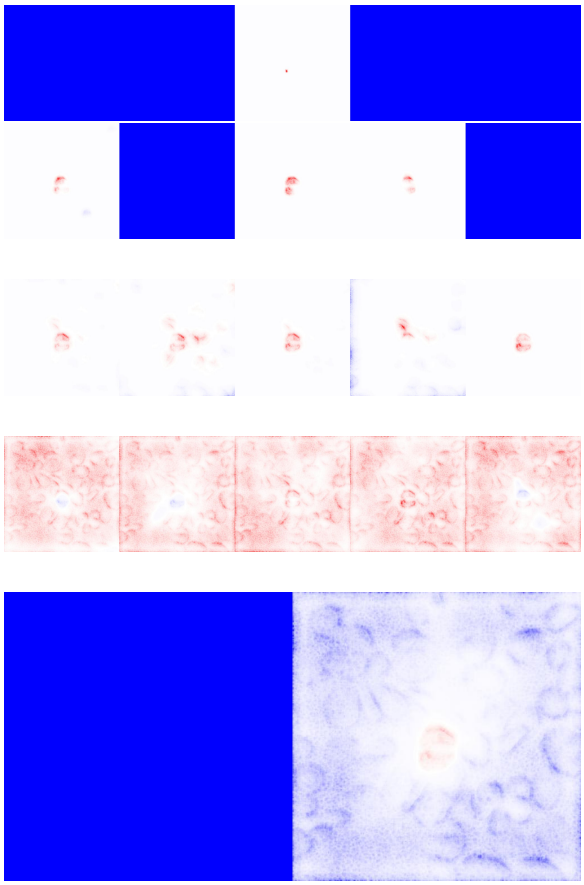

Abbildung 3: Konzept-Attributierungen der Layer features .8, features .18, features .22, classifier .3, classifier .6 für die Datei "Mitosis_152_{X=437293,Y=166519}.jpg"

## 3.2 Fragmentierung der Attributionen

Bezüglich der Dekomposition von Konzepten, die im latenten Raum ermittelt worden sind, zeigt Abb. 4, auf welchen unterliegenden Konzepte diese beruhen.

```
'model_ft.features.4': {31: 40, 30: 40, 28: 40, 29: 40, 25: 40},
'model_ft.features.8': {127: 40, 126: 40, 124: 40, 125: 40, 121: 40},
'model_ft.features.11': {255: 40, 254: 40, 252: 40, 253: 40, 249: 40},
'model_ft.features.15': {154: 15, 252: 14, 95: 13, 199: 10, 175: 10},
'model_ft.features.18': {359: 13, 460: 9, 406: 8, 93: 7, 40: 7},
'model_ft.features.22': {14: 20, 366: 18, 484: 16, 364: 16, 368: 16},
'model_ft.features.25': {286: 40, 418: 40, 335: 40, 114: 40, 366: 39},
'model_ft.classifier.0': {47: 40, 331: 40, 323: 40, 133: 40, 105: 40},
'model_ft.classifier.3': {2679: 40, 779: 40, 3439: 40, 4092: 40, 2424: 25}
```

Abbildung 4: Dictionary-Ausgabe der für die Konzepte wichtigsten Unterkonzepte, gemittelt über alle Samples

### 3.3 Bildregionenspezifische Attributionen

Anhand der auf dem Bildraum maskierten Heatmap konnte auch eine lokale Analyse ein Dictionary zu wichtigen Konzepten wiedergeben (Alg. 5).

```
'model_ft.features.0': {48: 29, 29: 29, 30: 29, 33: 28, 35: 27},
'model_ft.features.4': {52: 12, 55: 11, 25: 11, 13: 10, 20: 10},
'model_ft.features.8': {33: 14, 106: 12, 14: 11, 79: 11, 115: 9},
'model_ft.features.11': {183: 11, 154: 11, 199: 9, 229: 8, 242: 7},
'model_ft.features.15': {204: 12, 122: 10, 53: 10, 68: 10, 170: 9},
'model_ft.features.18': {76: 30, 12: 28, 256: 19, 68: 18, 366: 7},
'model_ft.features.22': {120: 39, 325: 27, 85: 23, 366: 20, 289: 19},
'model_ft.features.25': {323: 40, 193: 39, 17: 38, 26: 36, 105: 16},
'model_ft.classifier.0': {3257: 40, 949: 40, 2182: 40, 1421: 39, 1191: 29},
'model_ft.classifier.3': {3641: 40, 1091: 40, 863: 40, 3137: 40, 1464: 40}
```

Abbildung 5: Dictionary-Ausgabe der lokalen wichtigsten Konzepte, gemittelt über alle Samples

## 4 Fazit und Ausblick

Dieses "Independent Coursework II" hat die Klassifikation anhand von erlernten, in den Netzwerkparametern vorhandenen Konzepten untersucht. Hier haben sich mehrere Attributierungs-Methoden als sinnvoll für das Auffinden erwiesen. Bei der Untersuchung der Attributierungen fällt auf, dass besonders die mittleren und höheren Layer des untersuchten Netzwerks einige Konzepte enthalten, die ausschlaggebend für das Auffinden von Mitosen sind. Diese weisen hohe prozentuale Anteile über alle im jeweiligen Layer vorhandenen Konzepte auf. Dagegen scheinen in unteren Layern die Konzepte unwichtiger, jedoch werden über die Methoden hinweg die selben Encodings gemacht. Auch die exemplarische Untersuchung der Heatmaps zeigt, dass untere Layer weniger wichtig für die Mitosen-Klassifikation - und mittlere dagegen entscheidend sind.

Für zukünftige Arbeit wäre es interessant zu untersuchen, wie sich Konzepte respektive eines Multi-Klassifikators verhalten. Da über ein Netzwerk verstreut die Filter auch für ähnliche Klassen sprechen können, müsste untersucht werden, wo und wie diese Konzepte welche Klassenprädiktionen bestärken.

# Literatur


[1] S. Yeom, P. Seegerer, S. Lapuschkin, S. Wiedemann, K. Müller und W. Samek, „Pruning by Explaining: A Novel Criterion for Deep Neural Network Pruning“, *CoRR*, 2019. [Online]. Adresse: http://arxiv.org/abs/1912.08881.

[2] S. Mohammadjafari, M. Cevik, M. Thanabalasingam, A. Basar und A. D. N. Initiative, „Using ProtoPNet for Interpretable Alzheimer's Disease Classification“, *Proceedings of the Canadian Conference on Artificial Intelligence*, 8. Juni 2021, https://caiac.pubpub.org/pub/klwhoig4. DOI: 10.21428/594757db.fb59ce6c. [Online]. Adresse: https://caiac.pubpub.org/pub/klwhoig4.

[3] *Centrum für Biomedizinische Bild- und Informationsverarbeitung (CBMI)*, Accessed: 2010-09-30. [Online]. Adresse: https://cbmi.htw-berlin.de/.

[4] R. Achtibat *et al.*, „From" Where" to" What": Towards Human-Understandable Explanations through Concept Relevance Propagation“, *arXiv preprint arXiv:2206.03208*, 2022. [Online]. Adresse: https://arxiv.org/abs/2206.03208.

[5] R. Achtibat, M. Dreyer und S. Lapuschkin, *Zennit-CRP: An extension to Zennit using CRP*, 2022. [Online]. Adresse: https://github.com/rachtibat/zennit-crp.

[6] C. J. Anders, D. Neumann, W. Samek, K.-R. Müller und S. Lapuschkin, „Software for Dataset-wide XAI: From Local Explanations to Global Insights with Zennit, CoRelAy, and ViRelAy“, *CoRR*, Jg. abs/2106.13200, 2021.

[7] A. Paszke *et al.*, „PyTorch: An Imperative Style, High-Performance Deep Learning Library“, in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox und R. Garnett, Hrsg., Curran Associates, Inc., 2019, S. 8024–8035. [Online]. Adresse: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[8] C. R. Harris *et al.*, „Array programming with NumPy“, *Nature*, Jg. 585, Nr. 7825, S. 357–362, Sep. 2020. DOI: 10.1038/s41586-020-2649-2. [Online]. Adresse: https://doi.org/10.1038/s41586-020-2649-2.

[9] A. Clark, *Pillow (PIL Fork) Documentation*, 2015. [Online]. Adresse: https://buildmedia.readthedocs.org/media/pdf/pillow/latest/pillow.pdf.

[10] G. Bradski, „The OpenCV Library“, *Dr. Dobb's Journal of Software Tools*, 2000.

[11] I. Eisenbraun, *Concept Classification - ICW 2 (Github Repository)*, 2022. [Online]. Adresse: https://github.com/ilonae/Concept_Pruning_FP2.